Database Project Report Template

Due Date:

Project ID & Title: Digital Library & Open-Access Repository Management System

A. Project Identity

- **Team Name:** G8
- **Team Members:**

Nguyễn Trung Thiện (n25dcat107@gmail.student.ptithcm.edu.vn)

Trần Nguyễn Quốc Thịnh (n25dcat109@gmail.student.ptithcm.edu.vn)

Nguyễn Phúc Thịnh (n22dccn182@gmail.student.ptithcm.edu.vn)

- **Project Title:** Design and Implementation of a Relational Database for Academic Open-Access Digital Library and Repository System

B. Report structure

1. Introduction & Project Scope (Adapted from ISO/IEC/IEEE 29148)

1.1 System Objective

1.1.1 Problem Statement

Thực trạng quản lý tài liệu số và thư viện hiện nay:

Trong bối cảnh giáo dục hiện đại, các thư viện và trung tâm lưu trữ đang phải đối mặt với nhiều bất cập trong quá trình quản lý và vận hành hệ thống tài liệu. Sự phát triển đồng thời của tài liệu vật lý và tài liệu số đặt ra yêu cầu xây dựng một hệ thống quản lý dữ liệu thống nhất, có khả năng đáp ứng hiệu quả nhu cầu tra cứu, mượn tài liệu và truy cập các nguồn tài nguyên học thuật.

- **Sự phân mảnh giữa tài liệu vật lý và tài liệu số:** Nhiều thư viện hiện nay vẫn quản lý sách, giáo trình và tài liệu vật lý tách biệt với các tạp chí điện tử, bài báo khoa học và tài nguyên số. Việc sử dụng nhiều hệ thống khác nhau khiến dữ liệu bị phân tán, gây khó khăn cho người dùng trong quá trình tìm kiếm và tiếp cận tài liệu. Người dùng có thể phải thực hiện nhiều thao tác trên các nền tảng khác nhau để xác định vị trí, trạng thái hoặc quyền truy cập của một tài liệu.
- **Thiếu công cụ quản lý giấy phép Open Access:** Việc phát triển các kho tài liệu nghiên cứu mở (Open-Access Repository) giúp tăng khả năng tiếp cận tri thức. Tuy nhiên, nhiều hệ thống chưa có cơ chế quản lý thống nhất đối với các loại giấy phép sử dụng tài liệu. Điều này gây khó khăn trong việc xác định quyền truy cập,

sao chép, phân phối và trích dẫn tài liệu, đồng thời làm tăng nguy cơ sử dụng tài nguyên không đúng với điều kiện bản quyền.

- **Nghiệp vụ thư viện chồng chéo và phức tạp:**
    - Các nghiệp vụ như quản lý giới hạn số lượng tài liệu được mượn, thời hạn trả sách và xử lý tiền phạt quá hạn thường có nhiều quy tắc khác nhau tùy thuộc vào từng đối tượng người dùng. Nếu không được quản lý bằng một cơ sở dữ liệu thống nhất, thông tin dễ xảy ra sai lệch hoặc trùng lặp.
    - Một vấn đề đáng chú ý là việc quản lý các thực thể có nhiều vai trò trong hệ thống. Ví dụ, một người có thể đồng thời là **sinh viên hoặc giảng viên** với quyền mượn tài liệu, đồng thời là **tác giả** của các bài báo khoa học được lưu trữ trong kho tài liệu số. Nếu thiết kế cơ sở dữ liệu không hợp lý, thông tin cá nhân có thể bị lưu trùng lặp, gây khó khăn trong việc duy trì tính nhất quán và toàn vẹn dữ liệu.
- **Nhu cầu xây dựng hệ thống quản lý thống nhất:** Từ những vấn đề trên, cần xây dựng một hệ thống cơ sở dữ liệu có khả năng tích hợp quản lý tài liệu vật lý, tài liệu số, thông tin người dùng, quyền truy cập Open Access, hoạt động mượn – trả và tiền phạt trong cùng một hệ thống. Hệ thống cần đảm bảo dữ liệu được tổ chức khoa học, hạn chế trùng lặp, duy trì tính toàn vẹn và hỗ trợ hiệu quả các nghiệp vụ của thư viện.

1.1.2 Rationale

Lý do cần Open-Access Repository:

- Tạo ra một nền tảng tập trung vừa quản lý mượn trả sách vật lý, vừa lưu trữ và chia sẻ công khai các ấn phẩm nghiên cứu của trường đại học.
- Hỗ trợ các chuẩn bản quyền (như giấy phép Creative Commons) để bảo vệ quyền lợi tác giả đồng thời thúc đẩy việc chia sẻ tri thức khoa học minh bạch.
- Giải quyết các bài toán nghiệp vụ phức tạp như giới hạn mượn tài liệu, tính toán tiền phạt tự động và phân quyền truy cập.

1.1.3 System Objective

- **Trong phạm vi (In-scope):**

    ○ **Quản lý thành viên & Tác giả:** Quản lý thông tin người dùng (sinh viên, giảng viên, v.v.) và hồ sơ tác giả; giải quyết bài toán thực thể chồng chéo (một người vừa là thành viên mượn sách vừa là tác giả bài báo).

    ○ **Quản lý tài liệu vật lý:** Quản lý danh mục sách giấy truyền thống và các bản sao sách (Physical Books & Copies).

- **Quản lý nghiệp vụ mượn/trả:** Kiểm soát quy trình mượn - trả tài liệu và áp dụng các quy tắc kinh doanh về giới hạn mượn.
- **Quản lý tiền phạt:** Tính toán tự động và theo dõi trạng thái thu tiền phạt quá hạn.
- **Quản lý tài liệu số:** Lưu trữ và quản lý các tạp chí và bài báo điện tử (Journals/Articles).
- **Quản lý bản quyền mở:** Quản lý các loại giấy phép truy cập mở (Open-Access licenses, ví dụ: chuẩn Creative Commons).
- **Quản lý quyền truy cập:** Thiết lập quyền và trạng thái truy cập đối với các tài liệu số trong kho lưu trữ.

- **Ngoài phạm vi (Out-of-scope):**
  - **Tích hợp thanh toán trực tuyến:** Không phát triển cổng thanh toán tiền thật (chỉ dừng lại ở việc lưu trữ trạng thái "Đã nộp phạt" hoặc "Chưa nộp phạt" do thủ thư cập nhật).
  - **Quản lý nhân sự thư viện:** Không bao gồm phân hệ quản lý thông tin nhân viên, chấm công hay tính lương cho thủ thư.
  - **Quản lý kho vận logistics:** Không quản lý quy trình vận chuyển sách phức tạp giữa các kho bãi hoặc nhà cung cấp ngoài đời thực.

1.1.4 Project Objectives

- Xây dựng một cơ sở dữ liệu quan hệ hoàn chỉnh, tối ưu hóa để lưu trữ và truy vấn dữ liệu thư viện kết hợp kho lưu trữ mở.
- Áp dụng thành công các mô hình Extended ER (EER) đặc biệt là tính năng Chuyên môn hóa/Tổng quát hóa (Specialization/Generalization) để giải quyết các thực thể chồng chéo theo đúng yêu cầu của môn học.
- Hoàn thành tài liệu báo cáo tuần đúng hạn, tuân thủ cấu trúc chuẩn và các tiêu chuẩn tài liệu kỹ thuật.

1.1.5 Entity Overview

1.1.5 Phân hệ Thành viên & Tác giả

PERSON (Cá nhân)

Khái niệm: Là thực thể đại diện cho một con người cụ thể tương tác với hệ thống.

Lý do tồn tại: Được sử dụng làm thực thể cha (Supertype) trong mô hình EER để lưu trữ các thông tin định danh cơ bản như họ tên, email và số điện thoại, qua đó tránh trùng lặp dữ liệu khi một người đồng thời đảm nhận nhiều vai trò trong hệ thống.

MEMBER (Thành viên/Độc giả)

Khái niệm: Là cá nhân như sinh viên, giảng viên hoặc nghiên cứu sinh đã đăng ký và được cấp quyền sử dụng dịch vụ mượn, trả tài liệu vật lý tại thư viện.

Lý do tồn tại: Lưu trữ các thông tin nghiệp vụ liên quan đến việc mượn tài liệu như hạn mức mượn tối đa, ngày hết hạn thẻ và trạng thái tài khoản.

AUTHOR (Tác giả)

Khái niệm: Là cá nhân sáng tạo hoặc đóng góp nội dung cho các ấn phẩm khoa học, sách hoặc tài liệu được lưu trữ trong hệ thống.

Lý do tồn tại: Lưu trữ các thông tin học thuật của tác giả như đơn vị công tác, mã định danh ORCID và tiểu sử, độc lập với vai trò thành viên thư viện.

1.1.5 Phân hệ Tài liệu vật lý

PHYSICAL_BOOK (Đầu sách vật lý)

Khái niệm: Là thực thể đại diện cho một tác phẩm hoặc ấn phẩm xuất bản, chẳng hạn như giáo trình hoặc sách chuyên ngành.

Lý do tồn tại: Quản lý các thông tin xuất bản chung như ISBN, nhà xuất bản và năm xuất bản mà không phụ thuộc vào số lượng bản sao vật lý của đầu sách.

BOOK_COPY (Bản sao sách)

Khái niệm: Là một bản sao vật lý cụ thể của một đầu sách, tồn tại thực tế trong thư viện và có thể được mượn bởi thành viên.

Lý do tồn tại: Quản lý các thông tin riêng của từng bản sao như mã vạch (Barcode), vị trí lưu trữ và trạng thái sử dụng như sẵn sàng, đang được mượn, hỏng hoặc mất.

1.1.5 Phân hệ Tài liệu số & Lưu trữ mở

DIGITAL_ITEM (Tài liệu số)

Khái niệm: Là một ấn phẩm điện tử như bài báo khoa học, tạp chí, luận văn hoặc báo cáo nghiên cứu được lưu trữ trong kho tài liệu số.

Lý do tồn tại: Quản lý các thông tin của tài liệu số như đường dẫn tệp, DOI và trạng thái hoặc mức độ công khai của tài liệu.

LICENSE (Giấy phép bản quyền mở)

Khái niệm: Là tập hợp các điều khoản pháp lý quy định quyền truy cập, sao chép, phân phối hoặc sửa đổi đối với tài liệu số.

Lý do tồn tại: Quản lý các giấy phép Open Access, chẳng hạn như Creative Commons, nhằm xác định rõ quyền sử dụng tài liệu, bảo vệ quyền lợi của tác giả và hỗ trợ việc chia sẻ tri thức.

1.1.5 Phân hệ Mượn – Trả & Tiền phạt

BORROW_TRANSACTION (Giao dịch mượn – trả)

Khái niệm: Là thực thể ghi nhận một giao dịch trong đó MEMBER mượn một BOOK_COPY cụ thể trong một khoảng thời gian xác định.

Lý do tồn tại: Quản lý vòng đời của giao dịch mượn sách, bao gồm ngày mượn, hạn trả và ngày trả thực tế, đồng thời làm cơ sở xác định tình trạng quá hạn.

FINE (Tiền phạt)

Khái niệm: Là khoản nghĩa vụ tài chính phát sinh khi thành viên vi phạm quy định mượn – trả, chẳng hạn như trả sách quá hạn, làm hư hỏng hoặc làm mất sách.

Lý do tồn tại: Theo dõi số tiền phạt, ngày phát sinh và trạng thái thanh toán như chưa thanh toán hoặc đã thanh toán.

1.1.5 Phân hệ Liên kết & Phân loại

BOOK_AUTHOR (Liên kết Sách – Tác giả)

Khái niệm: Là thực thể liên kết giữa AUTHOR và PHYSICAL_BOOK, thể hiện mối quan hệ giữa tác giả và đầu sách.

Lý do tồn tại: Giải quyết quan hệ nhiều – nhiều (N) giữa AUTHOR và PHYSICAL_BOOK, đồng thời lưu trữ vai trò của tác giả đối với đầu sách như chủ biên, đồng tác giả hoặc dịch giả.

ITEM_AUTHOR (Liên kết Tài liệu số – Tác giả)

Khái niệm: Là thực thể liên kết giữa AUTHOR và DIGITAL_ITEM, thể hiện sự đóng góp của tác giả vào một tài liệu số.

Lý do tồn tại: Giải quyết quan hệ nhiều – nhiều (N) và lưu trữ các thông tin liên quan như thứ tự tác giả hoặc vai trò của tác giả trong tài liệu.

CATEGORY (Danh mục/Chủ đề)

Khái niệm: Là thực thể đại diện cho một lĩnh vực tri thức hoặc chủ đề chuyên môn dùng để phân loại tài liệu.

Lý do tồn tại: Hỗ trợ việc phân loại, tìm kiếm và duyệt tài liệu theo ngành học hoặc lĩnh vực nghiên cứu.

ITEM_CATEGORY (Liên kết Tài liệu – Danh mục)

Khái niệm: Là thực thể liên kết giữa DIGITAL_ITEM và CATEGORY, thể hiện các chủ đề mà một tài liệu số thuộc về.

Lý do tồn tại: Giải quyết quan hệ nhiều – nhiều (N), cho phép một tài liệu số thuộc nhiều danh mục hoặc một danh mục chứa nhiều tài liệu khác nhau.

1.2 Business Rules & Constraints

Hệ thống *Digital Library & Open-Access Repository Management System* vận hành dựa trên các quy tắc nghiệp vụ và ràng buộc toàn vẹn dữ liệu chuẩn hóa dưới đây:

[BR-01] Quy tắc Thực thể Cá nhân, Thành viên & Tác giả (Person, Member & Author - EER Overlap)

- **BR-01.1 (Identity & Single Storage):** Mỗi cá nhân tương tác với hệ thống được định danh duy nhất bởi person_id. Dữ liệu cá nhân (Họ tên, Email, Số điện thoại) chỉ được lưu trữ duy nhất một lần tại supertype PERSON.
- **BR-01.2 (Overlapping Roles):** Mối quan hệ giữa PERSON và hai subtype (MEMBER, AUTHOR) tuân thủ mô hình EER dạng **Overlapping** và **Partial Participation**. Một PERSON có thể chỉ là MEMBER, chỉ là AUTHOR, hoặc đồng thời đảm nhận cả hai vai trò.
- **BR-01.3 (Mandatory Inheritance):** Mọi bản ghi trong MEMBER và AUTHOR bắt buộc phải tham chiếu đến một person_id hợp lệ. Sự tồn tại độc lập không qua PERSON bị cấm hoàn toàn.
- **BR-01.4 (Unique Contact & Academic Identifiers):** Thuộc tính email trong PERSON và orcid trong AUTHOR (nếu có) phải là duy nhất trên toàn hệ thống (UNIQUE).

[BR-02] Quy tắc Quản lý Thành viên & Hạn mức Mượn (Member & Borrowing Limit Rules)

- **BR-02.1 (Account Status):** Trạng thái tài khoản thành viên chỉ nhận giá trị:

$$\text{membership_status} \in \{\text{'Active'},\text{'Expired'},\text{'Suspended'}\}$$

Active (Đang hoạt động): Thẻ thành viên còn thời hạn, tài khoản hợp lệ để thực hiện giao dịch mượn mới (khi đáp ứng đủ hạn mức mượn và không nợ tiền phạt).

Expired (Hết hạn): Thẻ thành viên đã vượt quá ngày hết hạn (current_date > expiry_date), tạm thời bị chặn mượn sách mới cho đến khi gia hạn thẻ.

Suspended (Tạm dừng/Đình chỉ): Tài khoản bị tạm khóa do vi phạm quy định (ví dụ: có khoản phạt Unpaid vượt ngưỡng cho phép của thư viện).

- **BR-02.2 (Role-based Max Limit):** Mỗi MEMBER được cấp một hạn mức mượn sách tối đa (max_borrow_limit > 0) tùy thuộc vào loại độc giả (ví dụ: *Sinh viên = 5*, *Giảng viên = 10*).
- **BR-02.3 (Borrowing Eligibility Check):** Thành viên chỉ được phép mở giao dịch mượn mới khi đáp ứng đồng thời 3 điều kiện:
    1. membership_status = 'Active' và thẻ còn hạn ($current_date \leq expiry_date$).
    2. Số lượng sách đang mượn thực tế hiện tại chưa đạt giới hạn:

$$\sum \text{Active_Loans} < \text{max_borrow_limit}$$

    3. Không có khoản nợ tiền phạt chưa thanh toán (fine_status = 'Unpaid') vượt quá ngưỡng quy định của thư viện.

[BR-03] Quy tắc Quản lý Sách Vật lý & Bản sao (Physical Book & Copy Rules)

- **BR-03.1 (Book & Copy Relationship):** Mỗi đầu sách PHYSICAL_BOOK có thể có 0 hoặc nhiều bản sao BOOK_COPY. Mỗi BOOK_COPY thuộc về đúng 01 PHYSICAL_BOOK và định danh bằng một mã vạch duy nhất barcode (UNIQUE).
- **BR-03.2 (Copy Status Domain):** Trạng thái bản sao chỉ nhận các giá trị cố định:

  copy_status ∈ {'Available','Borrowed','Lost','Damaged','In_Maintenance'}

  Available (Sẵn sàng): Bản sao đang có trên kệ/kho, đạt chất lượng và sẵn sàng cho mượn.

  Borrowed (Đang mượn): Bản sao đang nằm trong một giao dịch mượn chưa hoàn trả.

  Lost (Thất lạc/Mất): Bản sao được xác nhận bị mất trong quá trình mượn hoặc kiểm kê, kích hoạt khoản phạt tương ứng.

  In_Maintenance (Đang bảo trì): Bản sao đang trong quá trình sửa chữa, phục chế, tạm thời không phục vụ cho mượn.

  Damaged (Hư hỏng): Bản sao bị ghi nhận hỏng/rách khi trả sách hoặc kiểm kê.
- **BR-03.3 (Availability Enforcement):** Chỉ những BOOK_COPY có copy_status = 'Available' mới được phép khởi tạo giao dịch mượn mới.

[BR-04] Quy tắc Nghiệp vụ Mượn - Trả & Chuyển đổi Trạng thái (Borrowing Transaction Rules)

- **BR-04.1 (Active Loan Uniqueness):** Tại một thời điểm, một BOOK_COPY chỉ được nằm trong tối đa 01 giao dịch mượn đang hoạt động (giao dịch có return_date IS NULL).
    - **Giao dịch đang hoạt động (Active Loan):** Giá trị return_date IS NULL (chưa trả sách). Mỗi bản sao sách tại một thời điểm chỉ cho phép tối đa 01 giao dịch mượn đang hoạt động.
    - **Giao dịch đã hoàn tất:** Đã ghi nhận ngày trả thực tế vào trường return_date.
- **BR-04.2 (Temporal Integrity Constraints):** Thời gian giao dịch phải tuân thủ các ràng buộc logic:

$$borrow_date \leq due_date \quad \text{và} \quad borrow_date \leq return_date$$

- **BR-04.3 (Automatic State Transition):**
    - Khi tạo giao dịch mượn, copy_status tự động chuyển từ 'Available' → 'Borrowed'.
    - Khi trả sách thành công và kiểm tra không hư hại, copy_status chuyển từ 'Borrowed' → 'Available'.
    - Nếu ghi nhận hư hỏng hoặc mất mát khi trả, copy_status chuyển tương ứng sang 'Damaged' hoặc 'Lost'.

[BR-05] Quy tắc Tính Tiền phạt & Lưu vết Tài chính (Fine & Penalty Rules)

- **BR-05.1 (Fine Trigger Conditions):** Khoản phạt FINE được tự động khởi tạo khi $return_date > due_date$ (trả quá hạn), hoặc khi BOOK_COPY bị ghi nhận 'Damaged' / 'Lost'.
- **BR-05.2 (Overdue Fine Calculation):** Số tiền phạt $fine_amount \geq 0$. Đối với vi phạm quá hạn, tiền phạt được tự động tính:

$$fine_amount = (return_date - due_date) \times daily_penalty_rate$$

- **BR-05.3 (Payment Audit & System Scope):**
    - Trạng thái phạt fine_status ∈ {'Unpaid','Paid'}. Khi fine_status = 'Paid', hệ thống bắt buộc lưu vết ngày thanh toán payment_date.
        - **Unpaid (Chưa thanh toán):** Khoản phạt đã phát sinh nhưng chưa được nộp.

        - **Paid (Đã thanh toán):** Khoản phạt đã được độc giả nộp và thủ thư cập nhật xác nhận trên hệ thống, bắt buộc kèm theo thông tin ngày thanh toán payment_date.
    - Hệ thống chỉ ghi nhận trạng thái thanh toán do thủ thư cập nhật; **không** tích hợp cổng thanh toán trực tuyến tài chính.
    - Tuyệt đối không xóa bản ghi FINE khỏi CSDL để bảo đảm tính minh bạch khi đối soát tài chính.

[BR-06] Quy tắc Tài liệu Số, Giấy phép & Embargo (Digital Item & Open Access Rules)

- **BR-06.1 (Digital Identifiers & Storage):** Mỗi DIGITAL_ITEM bắt buộc có đường dẫn lưu trữ (file_path) và định dạng tệp. Mã định danh số doi (nếu có) phải là duy nhất trên toàn hệ thống (UNIQUE).
- **BR-06.2 (License Requirement):** Mỗi DIGITAL_ITEM bắt buộc phải liên kết với đúng 01 loại giấy phép LICENSE (ví dụ: CC BY 4.0, CC BY-NC 4.0, Public Domain) để xác định quyền sử dụng.
- **BR-06.3 (Access Policy & Embargo Period):** Mức độ truy cập tài liệu số thuộc tập:

  access_level ∈ {'Public_Open_Access','Restricted_Campus_Only','Embargoed'}

  Phân quyền và quản lý chính sách truy cập đối với các tài liệu, bài báo trong kho lưu trữ mở:

  Public_Open_Access (Mở công khai): Tài liệu được chia sẻ hoàn toàn tự do cho mọi người dùng xem và tải xuống.

  Restricted_Campus_Only (Giới hạn nội bộ): Tài liệu chỉ dành riêng cho người dùng thuộc nội bộ khuôn viên trường/viện nghiên cứu.

  Embargoed (Khóa tạm thời): Tài liệu tạm thời bị khóa truy cập công khai theo thỏa thuận bản quyền; tự động/được phép chuyển sang mở công khai khi qua ngày hết hạn embargo (current_date > embargo_end_date).

  Tài liệu bị khóa tạm thời ('Embargoed') chỉ được phép chuyển sang công khai khi $current_date > embargo_end_date$.

[BR-07] Quy tắc Phân loại & Quyền Tác giả (Authorship & Categorization Rules)

- **BR-07.1 (Many-to-Many Mapping):** Quan hệ Nhiều - Nhiều (N:M) giữa Tác giả với Sách vật lý và Tài liệu số được quản lý lần lượt qua các bảng liên kết BOOK_AUTHOR và ITEM_AUTHOR.

- **BR-07.2 (Authorship Metadata):**
    - BOOK_AUTHOR bắt buộc lưu vai trò của tác giả (role ∈ {'Main Author','Co-Author','Editor','Translator'}).
    - ITEM_AUTHOR bắt buộc lưu thứ tự đóng góp (author_order ≥ 1) và vai trò tác giả trong bài báo/tạp chí số.
- **BR-07.3 (Categorization):** Mối quan hệ N:M giữa DIGITAL_ITEM và CATEGORY được quản lý qua bảng liên kết ITEM_CATEGORY.

[BR-08] Ràng buộc Toàn vẹn Kỹ thuật CSDL (Database Referential Integrity Rules)

- **BR-08.1 (Primary & Foreign Keys):** Toàn bộ Khóa chính (PK) phải duy nhất và NOT NULL. Khóa ngoại (FK) phải tham chiếu đến bản ghi cha hợp lệ đang tồn tại.
- **BR-08.2 (Deletion Constraints):** Sử dụng ON DELETE RESTRICT đối với các bảng nghiệp vụ mượn/trả và tiền phạt để chống xóa nhầm lịch sử dữ liệu; sử dụng ON DELETE CASCADE đối với các bảng liên kết trung gian (như BOOK_AUTHOR, ITEM_CATEGORY) khi thực thể chính bị xóa.

2. Database Design (ISO/IEC 19505 / IE Standards)

2.1 Conceptual Model (ER/EER Diagram)

Mô tả Mô hình Thực thể - Liên kết Mở rộng (EER Model Description)

Mô hình EER của hệ thống **Digital Library & Open-Access Repository Management System** được thiết kế để bao quát toàn bộ quy trình nghiệp vụ từ quản lý sách truyền thống đến các tài nguyên nghiên cứu mở. Mô hình giải quyết các điểm cốt lõi sau:

1. **Mối quan hệ Kế thừa & Chồng chéo (Specialization/Generalization với Overlap):**
    - **Supertype:** PERSON (Lưu thông tin định danh dùng chung: person_id, first_name, last_name, email, phone_number).
    - **Subtypes:** MEMBER và AUTHOR.
    - **Ràng buộc EER:**
        - Dạng **Overlapping** : Một cá nhân có thể chỉ là độc giả (MEMBER), chỉ là tác giả (AUTHOR), hoặc đồng thời vừa là MEMBER vừa là AUTHOR.
        - Dạng **Partial Participation**: Một PERSON có thể tồn tại mà không bắt buộc phải nằm trong MEMBER hay AUTHOR (ví dụ: người dùng đăng ký tài khoản khách chờ duyệt).
2. **Sách vật lý (Physical Books & Copies):**

- Quan hệ 1:N giữa PHYSICAL_BOOK và BOOK_COPY: Mỗi đầu sách có thể có 0 hoặc nhiều bản sao vật lý. Mỗi bản sao lưu giữ mã vạch barcode duy nhất và trạng thái copy_status.

3. **Giao dịch & Tiền phạt (Borrowing & Fines):**
    - Quan hệ 1:N giữa MEMBER và BORROW_TRANSACTION: Một thành viên thực hiện nhiều giao dịch mượn sách.
    - Quan hệ 1:N giữa BOOK_COPY và BORROW_TRANSACTION: Một bản sao sách có thể được mượn qua nhiều giao dịch theo thời gian.
    - Quan hệ 1:1 (hoặc 1:0..1) giữa BORROW_TRANSACTION và FINE: Một giao dịch mượn nếu vi phạm quá hạn hoặc hư hỏng sẽ phát sinh tối đa 01 khoản phạt FINE.
4. **Kho tài liệu số & Bản quyền mở (Digital Repository & Licensing):**
    - Quan hệ 1:N giữa LICENSE và DIGITAL_ITEM: Một loại giấy phép truy cập mở (như CC BY 4.0, Public Domain) có thể áp dụng cho nhiều tài liệu số.
    - Quan hệ N:M giữa DIGITAL_ITEM và CATEGORY: Một tài liệu số có thể thuộc nhiều danh mục/chủ đề và ngược lại (thông qua thực thể liên kết ITEM_CATEGORY).
5. **Liên kết Tác giả (Authorship Mapping):**
    - Quan hệ N:M giữa AUTHOR và PHYSICAL_BOOK: Quản lý thông tin tác giả viết sách qua BOOK_AUTHOR (lưu vai trò: Main Author, Co-Author, Editor, v.v.).
    - Quan hệ N:M giữa AUTHOR và DIGITAL_ITEM: Quản lý thông tin tác giả bài báo/tạp chí qua ITEM_AUTHOR (lưu thứ tự tác giả author_order và vai trò).

2.2 Logical Schema Mapping (Chuyển đổi sang Lược đồ Quan hệ)

1. Bảng PERSON (Thực thể Cha - Quản lý Con người)

Lưu trữ thông tin cốt lõi dùng chung cho tất cả các đối tượng tham gia hệ thống.

Tên trường (Column)	Kiểu dữ liệu (Data Type)	Ràng buộc (Constraint)	Mô tả (Description)
person_id	INT	PRIMARY KEY, AUTO_INCREMENT	Mã định danh cá nhân
first_name	VARCHAR (50)	NOT NULL	Tên
last_name	VARCHAR (50)	NOT NULL	Họ
email	VARCHAR (100)	UNIQUE, NOT NULL	Email liên hệ
phone_number	VARCHAR (20)	NULLABLE	Số điện thoại
created_at	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP	Thời điểm tạo hồ sơ

2. Bảng MEMBER (Lớp con cấp 1 - Độc giả thư viện)

Lớp con kế thừa từ PERSON, quản lý thông tin thành viên thư viện nói chung.

Tên trường (Column)	Kiểu dữ liệu (Data Type)	Ràng buộc (Constraint)	Mô tả (Description)
person_id	INT	PRIMARY KEY, FK -> PERSON(person_id) ON DELETE CASCADE	Khóa chính kế thừa từ lớp cha
member_code	VARCHAR (20)	UNIQUE, NOT NULL	Mã thẻ thành viên (duy nhất)
membership_status	VARCHAR (20)	NOT NULL	Trạng thái thẻ (Active, Suspended, Expired)

Tên trường (Column)	Kiểu dữ liệu (Data Type)	Ràng buộc (Constraint)	Mô tả (Description)
max_borrow_limit	INT	DEFAULT 5	Giới hạn số sách mượn tối đa (> 0)
expiry_date	Date	NOT NULL	Ngày hết hạn thẻ thành viên

3. Bảng AUTHOR

Tên trường (Column)	Kiểu dữ liệu (Data Type)	Ràng buộc (Constraint)	Mô tả (Description)
person_id	INT	PRIMARY KEY, FOREIGN KEY (PERSON.person_id)	Mã tác giả(liên kết từ bảng PERSON)
author_code	VARCHAR(20)	UNIQUE, NOT NULL	Mã số tác giả (duy nhất)
affiliation	VARCHAR(100)	NULLABLE	Đơn vị/Cơ quan công tác
orcid	VARCHAR(20)	UNIQUE, NULLABLE	Mã số định danh tác giả quốc tế
bio	TEXT	NULLABLE	Tiểu sử tác giả

4. Bảng PHYSICAL_BOOK

Tên trường (Column)	Kiểu dữ liệu (Data Type)	Ràng buộc (Constraint)	Mô tả (Description)
book_id	INT	PRIMARY KEY, AUTO_INCREMENT	Mã định danh đầu sách vật lý
Title	VARCHAR(255)	NOT NULL	Tên/tiêu đề sách
isbn	VARCHAR(20)	UNIQUE, NOT NULL	Mã số tiêu chuẩn quốc tế ISBN
publisher	VARCHAR(100)	NULLABLE	Nhà xuất bản
publication_year	INT	NULLABLE	Năm xuất bản

5. Bảng BOOK_COPY

Tên trường (Column)	Kiểu dữ liệu (Data Type)	Ràng buộc (Constraint)	Mô tả (Description)
copy_id	INT	PRIMARY KEY, AUTO_INCREMENT	Mã bản sao cuốn sách
book_id	INT	FOREIGN KEY (PHYSICAL_BOOK.book_id)	Mã đầu sách
barcode	VARCHAR(50)	UNIQUE, NOT NULL	Mã vạch quản lý bản sao
copy_status	VARCHAR(20)	NOT NULL DEFAULT 'Available'	Trạng thái bản sao (Available, Borrowed, Lost, Damaged, In_Maintenance)

Tên trường (Column)	Kiểu dữ liệu (Data Type)	Ràng buộc (Constraint)	Mô tả (Description)
shelf_location	VARCHAR(50)	NULLABLE	Vị trí lưu trữ trên kệ

6. Bảng DIGITAL_ITEM (Thực thể Cha - Tài nguyên số)

Tên trường (Column)	Kiểu dữ liệu (Data Type)	Ràng buộc (Constraint)	Mô tả (Description)
item_id	INT	PRIMARY KEY, AUTO_INCREMENT	Mã định danh tài nguyên
License_id	Int	FOREIGN KEY (LICENSE.license_id), NOT NULL	Mã giấy phép sử dụng
title	TEXT	NOT NULL	Tiêu đề tài liệu
doi	VARCHAR (100)	UNIQUE, NULLABLE	Mã định danh DOI quốc tế
file_path	String	NOT NULL	Đường dẫn lưu trữ tệp tin
access_level	VARCHAR (50)	NOT NULL	Quyền truy cập (Open, Embargo, Restricted)
embargo_end_date	DATE	NULLABLE	Ngày hết hạn khóa tạm thời của một tài liệu số

7. Bảng BORROW_TRANSACTION

Tên trường (Column)	Kiểu dữ liệu (Data Type)	Ràng buộc (Constraint)	Mô tả (Description)
transaction_id	INT	PRIMARY KEY, AUTO_INCREMENT	Mã giao dịch mượn trả
Member_id	INT	FOREIGN KEY (MEMBER.person_id), NOT NULL	Mã thành viên mượn
Copy_id	INT	FOREIGN KEY (BOOK_COPY.copy_id), NOT NULL	Mã bản sao cuốn sách được mượn
Borrow_date	Date	NOT NULL	Ngày thực hiện mượn
Due_date	Date	NOT NULL	Ngày hẹn trả sách
Return_date	Date	NULLABLE	Ngày thực tế trả sách

8. Bảng Fine

Tên trường (Column)	Kiểu dữ liệu (Data Type)	Ràng buộc (Constraint)	Mô tả (Description)
fine_id	INT	PRIMARY KEY, AUTO_INCREMENT	Mã khoản phạt

Tên trường (Column)	Kiểu dữ liệu (Data Type)	Ràng buộc (Constraint)	Mô tả (Description)
Transaction_id	INT	FOREIGN KEY (BORROW_TRANSACTION.transaction_id), NOT NULL	Mã giao dịch mượn liên quan
Fine_amount	DECIMAL(10, 2)	NOT NULL	Số tiền phạt
Fine_reason	VARCHAR(255)	NULLABLE	Lý do phạt
Fine_status	VARCHAR(20)	NOT NULL	Trạng thái nộp phạt
Payment_date	Date	NULLABLE	Ngày hoàn tất thanh toán tiền phạt

9 Bảng LICENSE

Tên trường (Column)	Kiểu dữ liệu (Data Type)	Ràng buộc (Constraint)	Mô tả (Description)
license_id	INT	PRIMARY KEY, AUTO_INCREMENT	Mã bản quyền/giấy phép
License_name	VARCHAR(100)	NOT NULL	Tên bản quyền/giấy phép

Tên trường (Column)	Kiểu dữ liệu (Data Type)	Ràng buộc (Constraint)	Mô tả (Description)
License_code	VARCHAR(50)	UNIQUE, NOT NULL	Mã ký hiệu bản quyền

10 Bảng CATEGORY

Tên trường (Column)	Kiểu dữ liệu (Data Type)	Ràng buộc (Constraint)	Mô tả (Description)
Category_id	INT	PRIMARY KEY, AUTO_INCREMENT	Mã danh mục/phân loại
Category_name	VARCHAR(100)	UNIQUE, NOT NULL	Tên danh mục/thể loại

11 Bảng BOOK_AUTHOR

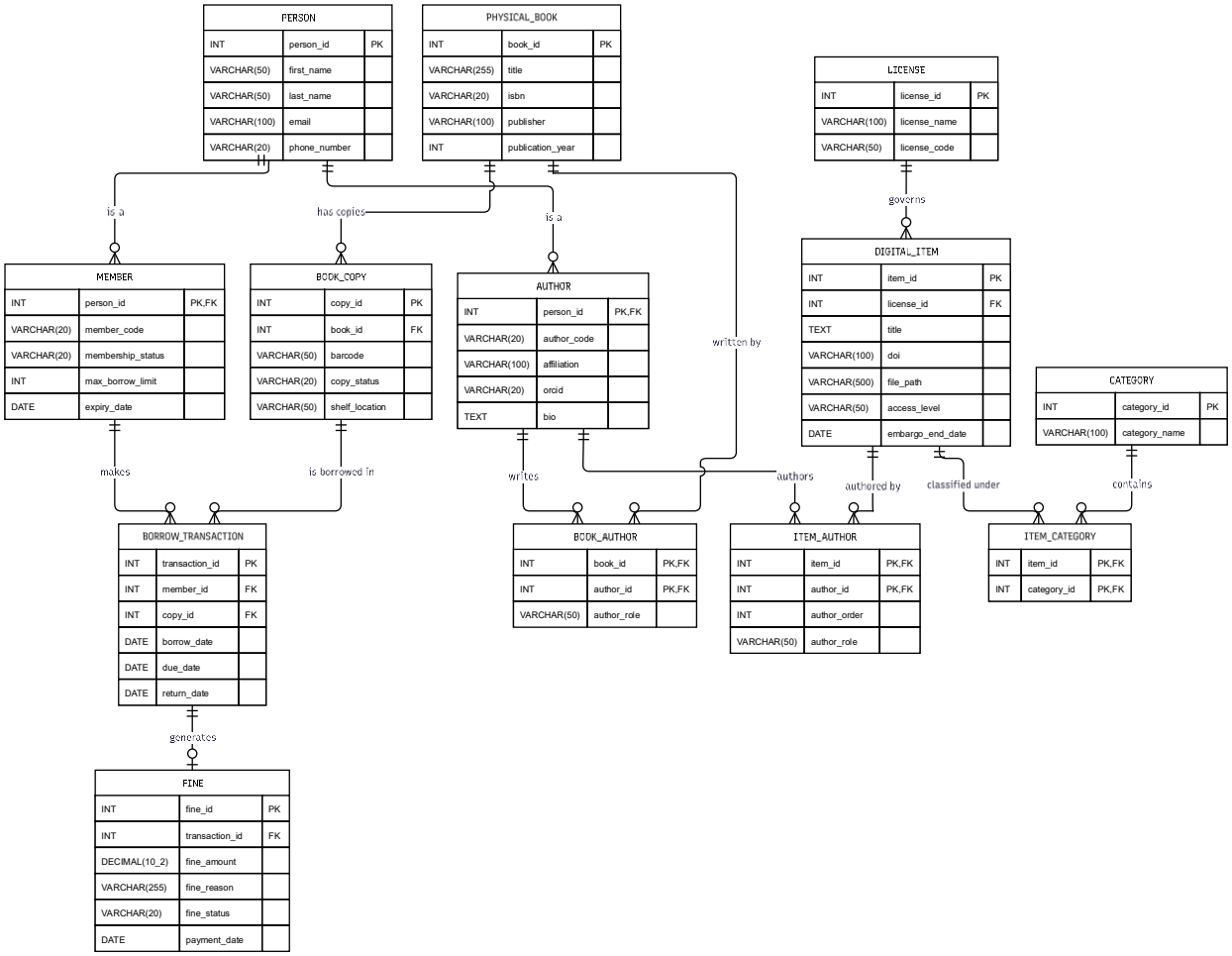
Tên trường (Column)	Kiểu dữ liệu (Data Type)	Ràng buộc (Constraint)	Mô tả (Description)
book_id	INT	PRIMARY KEY, FOREIGN KEY (PHYSICAL_BOOK.book_id)	Mã sách vật lý
Author_id	Int	PRIMARY KEY, FOREIGN KEY (AUTHOR.person_id)	Mã tác giả
Author_role	VARCHAR(50)	NULLABLE	Vai trò tác giả

12 Bảng ITEM_AUTHOR

Tên trường (Column)	Kiểu dữ liệu (Data Type)	Ràng buộc (Constraint)	Mô tả (Description)
Item_id	INT	PRIMARY KEY, FOREIGN KEY (DIGITAL_ITEM.item_id)	Mã tài liệu số
Author_id	INT	PRIMARY KEY, FOREIGN KEY (AUTHOR.person_id)	Mã tác giả
Author_order	INT	NULLABLE	Thứ tự sắp xếp tên tác giả
Author_role	VARCHAR(50)	NULLABLE	Vai trò tác giả đối với tài liệu số

13 Bảng ITEM_CATEGORY

Tên trường (Column)	Kiểu dữ liệu (Data Type)	Ràng buộc (Constraint)	Mô tả (Description)
item_id	INT	PRIMARY KEY, FOREIGN KEY (DIGITAL_ITEM.item_id)	Mã tài liệu số
Category_id	INT	PRIMARY KEY, FOREIGN KEY (CATEGORY.category_id)	Mã danh mục



2.3 Normalization Verification (Chứng minh chuẩn hóa 1NF -> BCNF)

Quá trình chuẩn hóa dữ liệu đảm bảo loại bỏ hiện tượng dư thừa dữ liệu (Data Redundancy) và ngăn ngừa các bất thường khi thao tác dữ liệu (Insertion, Deletion, Update Anomalies). Dưới đây là chứng minh chi tiết cho từng thực thể trong hệ thống.

2.3.1 Các Phụ thuộc hàm cốt lõi trong Hệ thống (Core Functional Dependencies)

Dựa trên các quy tắc nghiệp vụ (BR-01 đến BR-08), tập hợp các Phụ thuộc hàm (FD) của từng thực thể được xác định như sau:

1. **PERSON:**
    - FD1: person_id -> first_name, last_name, email, phone_number, created_at
    - FD2: email -> person_id, first_name, last_name, phone_number, created_at (Do email là Unique)
2. **MEMBER:**

- FD1: person_id -> member_code, membership_status, max_borrow_limit, expiry_date
- FD2: member_code -> person_id, membership_status, max_borrow_limit, expiry_date

3. **AUTHOR:**
   - FD1: person_id -> author_code, affiliation, orcid, bio
   - FD2: orcid -> person_id, author_code, affiliation, bio
4. **PHYSICAL_BOOK**:
   - FD1: book_id -> title, isbn, publisher, publication_year
   - FD2: isbn -> book_id, title, publisher, publication_year
5. **BOOK_COPY:**
   - FD1: copy_id -> barcode, copy_status, shelf_location, book_id
   - FD2: barcode -> copy_id, copy_status, shelf_location, book_id
6. **BORROW_TRANSACTION:**
   - FD1: transaction_id -> borrow_date, due_date, return_date, member_id, copy_id
7. **FINE:**
   - FD1: fine_id -> fine_amount, fine_reason, fine_status, payment_date, transaction_id
   - FD2: transaction_id -> fine_id, fine_amount, fine_reason, fine_status, payment_date (Do quan hệ 1:0..1)
8. **DIGITAL_ITEM:**
   - FD1: item_id -> title, abstract, doi, file_path, file_format, access_level, embargo_end_date, license_id
   - FD2: doi -> item_id, title, abstract, file_path, file_format, access_level, embargo_end_date, license_id
9. **LICENSE & CATEGORY:**
   - FD1: license_id -> license_name, license_code, terms_description
   - FD1: category_id -> category_name, description

10. **ASSOCIATIVE ENTITIES (BOOK_AUTHOR, ITEM_AUTHOR, ITEM_CATEGORY):**
    - (book_id, author_id) -> author_role
    - (item_id, author_id) -> author_order, author_role
    - (item_id, category_id) -> Rong (Chỉ chứa thuộc tính khóa)

2.3.2 Quy trình Chứng minh Các Dạng Chuẩn (Normalization Steps)

1. **Dạng chuẩn 1 (First Normal Form - 1NF)**
    - **Định nghĩa:** Một quan hệ thuộc 1NF nếu tất cả các thuộc tính đều chứa các giá trị đơn lẻ (Atomic values) và không có tập thuộc tính lặp (Repeating groups).
    - **Chứng minh:**
        - Toàn bộ các thuộc tính phức hợp (Composite Attributes) đã được phân rã thành các thuộc tính đơn (Ví dụ: Họ tên người dùng được tách thành first_name và last_name thay vì lưu chung full_name).
        - Không có thuộc tính đa trị (Multi-valued Attributes) (Ví dụ: Một tài liệu số có nhiều danh mục không lưu dạng chuỗi phân cách category_ids = "1, 2, 3" trong DIGITAL_ITEM, mà được tách riêng ra bảng trung gian ITEM_CATEGORY).
        - Tất cả các trường dữ liệu đều có kiểu dữ liệu nguyên tố (INTEGER, VARCHAR, DATETIME, DECIMAL).
    - Kết luận: Tất cả các bảng trong thiết kế đều Đạt 1NF.
2. **Dạng chuẩn 2 (Second Normal Form - 2NF)**
    - **Định nghĩa:** Một quan hệ thuộc 2NF nếu nó đã đạt 1NF và không tồn tại phụ thuộc hàm một phần (Partial Dependency). Nghĩa là mọi thuộc tính không khóa (Non-prime attribute) phải phụ thuộc đầy đủ vào toàn bộ Khóa chính (Primary Key).
    - **Chứng minh:**
        - Đối với các bảng có Khóa chính đơn (1 thuộc tính): PERSON, MEMBER, AUTHOR, PHYSICAL_BOOK, BOOK_COPY, BORROW_TRANSACTION, FINE, LICENSE, DIGITAL_ITEM, CATEGORY.

- **Lý thuyết:** Do Khóa chính chỉ gồm 1 thuộc tính nên không thể tồn tại phụ thuộc hàm vào một phần của khóa. Do đó, các bảng này nghiễm nhiên đạt 2NF.
- Đối với các bảng có Khóa chính hợp phần (Composite Primary Key):
  - BOOK_AUTHOR với PK = (book_id, author_id): Thuộc tính không khóa author_role phụ thuộc vào cả hợp phần (book_id, author_id) chứ không phụ thuộc riêng vào book_id hay author_id.
  - ITEM_AUTHOR với PK = (item_id, author_id): Các thuộc tính không khóa author_order và author_role phụ thuộc vào cả hợp phần (item_id, author_id).
  - ITEM_CATEGORY với PK = (item_id, category_id): Không có thuộc tính không khóa.

- **Kết luận:** Không tồn tại phụ thuộc hàm một phần trong bất kỳ thực thể nào. Tất cả đều Đạt 2NF.

3. **Dạng chuẩn 3 (Third Normal Form - 3NF)**
   - Định nghĩa: Một quan hệ thuộc 3NF nếu nó đã đạt 2NF và không tồn tại phụ thuộc hàm bắc cầu (Transitive Dependency). Nghĩa là không có thuộc tính không khóa nào phụ thuộc vào một thuộc tính không khóa khác (X -> Y -> Z).
   - **Chứng minh:**
     - Xét bảng BOOK_COPY: Thuộc tính book_id là Foreign Key trỏ tới PHYSICAL_BOOK. Các thông tin của sách như title, publisher, isbn được lưu hoàn toàn ở bảng PHYSICAL_BOOK, không lưu lặp lại ở BOOK_COPY. Nhờ đó loại bỏ phụ thuộc bắc cầu copy_id -> book_id -> title.
     - Xét bảng BORROW_TRANSACTION: Không lưu thông tin tên độc giả hay tên sách. Phụ thuộc bắc cầu transaction_id -> member_id -> email bị triệt tiêu do email được lưu tại PERSON.
     - Xét bảng DIGITAL_ITEM: Thuộc tính license_id là Foreign Key. Các chi tiết giấy phép (license_name, terms_description) được tách riêng ra bảng LICENSE, loại bỏ phụ thuộc bắc cầu item_id -> license_id -> terms_description.
     - Xét bảng FINE: Tiền phạt gắn liền với giao dịch mượn qua transaction_id. Thông tin người nợ phạt được truy vết qua BORROW_TRANSACTION -> MEMBER, không lưu trực tiếp member_id ở bảng FINE.

- Kết luận: Tất cả các thuộc tính không khóa trong mọi bảng đều phụ thuộc trực tiếp và duy nhất vào Khóa chính/Khóa ứng viên. Tất cả đều Đạt 3NF.

4. Dạng chuẩn Boyce-Codd (Boyce-Codd Normal Form - BCNF)
    - Định nghĩa: Một quan hệ thuộc BCNF nếu nó đã đạt 3NF và với mọi phụ thuộc hàm có dạng X -> Y (với Y không phải tập con của X), thì X phải là một Superkey (hoặc Candidate Key).
    - Chứng minh:
        - Xét các Phụ thuộc hàm có Vế trái (X) không phải Khóa chính ban đầu:
            - Trong PERSON: FD2: email -> ... => email là một Khóa ứng viên (Candidate Key / Unique constraint).
            - Trong MEMBER: FD2: member_code -> ... => member_code là Khóa ứng viên.
            - Trong AUTHOR: FD2: orcid -> ... => orcid là Khóa ứng viên.
            - Trong PHYSICAL_BOOK: FD2: isbn -> ... => isbn là Khóa ứng viên.
            - Trong BOOK_COPY: FD2: barcode -> ... => barcode là Khóa ứng viên.
            - Trong FINE: FD2: transaction_id -> ... => transaction_id là Khóa ứng viên (do quan hệ 1:0..1).
            - Trong DIGITAL_ITEM: FD2: doi -> ... => doi là Khóa ứng viên.
        - Vì tất cả các vế trái (X) của toàn bộ tập Phụ thuộc hàm trong hệ thống đều là Khóa chính (Primary Key) hoặc Khóa ứng viên (Candidate Key), nên thiết kế không vi phạm bất kỳ điều kiện BCNF nào.
    - Kết luận: Toàn bộ cơ sở dữ liệu Đạt dạng chuẩn BCNF.

3. Data Dictionary (Adapted from ISO/IEC 11179)

Từ điển dữ liệu dưới đây tuân thủ chuẩn ISO/IEC 11179 về quản lý thông tin tả dữ liệu (Metadata Registry). Tất cả các tên bảng và tên thuộc tính sử dụng định dạng snake_case đồng bộ với lược đồ quan hệ tại Mục 2.2 và quy tắc SQLStyle.guide.

3.1 Bảng person (Thực thể cha Supertype)

Attribute Name	Data Type	Key Type	Nullable ?	Default Value	Business Rules / Constraints
person_id	INT	PK	No	Auto Increment	Khóa chính tự động tăng, định danh duy nhất mỗi cá nhân.
first_name	VARCHAR(50)	None	No	None	Tên của cá nhân, không được để trống.
last_name	VARCHAR(50)	None	No	None	Họ và tên đệm, không được để trống.
email	VARCHAR(100)	Candidate Key	No	None	Email duy nhất (UNIQUE), đúng định dạng địa chỉ email. [BR-01.4]
phone_number	VARCHAR(20)	None	Yes	NULL	Số điện thoại liên hệ cá nhân.
created_at	TIMESTAMP	None	No	CURRENT_TIMESTAMP	Thời điểm tạo hồ sơ cá nhân trong hệ thống.

3.2 Bảng member (Thực thể con Subtype - Độc giả)

Attribute Name	Data Type	Key Type	Nullable ?	Default Value	Business Rules / Constraints
person_id	INT	PK, FK	No	None	Khóa chính kế thừa từ

					person(person_id). ON DELETE CASCADE. [BR-01.3]
member_code	VARCHAR(20)	Candidate Key	No	None	Mã thẻ thành viên duy nhất (UNIQUE), ví dụ: 'MEM2026001'.
membership_status	VARCHAR(20)	None	No	'Active'	Giá trị chỉ thuộc: 'Active', 'Expired', 'Suspended'. [BR-02.1]
max_borrow_limit	INT	None	No	5	Hạn mức mượn tối đa, phải lớn hơn 0 (CHECK (max_borrow_limit > 0)). [BR-02.2]
expiry_date	DATE	None	No	None	Ngày hết hạn thẻ thành viên.

3.3 Bảng author (Thực thể con Subtype - Tác giả)

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
person_id	INT	PK, FK	No	None	Khóa chính kế thừa từ person(person_id). ON DELETE CASCADE. [BR-01.3]
author_code	VARCHAR(20)	Candidate Key	No	None	Mã số tác giả duy nhất (UNIQUE), ví dụ: 'AUT2026001'.
affiliation	VARCHAR(100)	None	Yes	NULL	Cơ quan, trường đại học hoặc đơn vị công tác của tác giả.
orcid	VARCHAR(20)	Candidate Key	Yes	NULL	Mã định danh tác giả quốc tế duy nhất (UNIQUE), chuẩn ORCID iD. [BR-01.4]
bio	TEXT	None	Yes	NULL	Tóm tắt tiểu sử công tác và hướng nghiên cứu của tác giả.

3.4 Bảng physical_book (Đầu sách vật lý)

Attribute Name	Data Type	Key Type	Nullable ?	Default Value	Business Rules / Constraints
book_id	INT	PK	No	Auto Increment	Mã định danh duy nhất của đầu sách vật lý.
title	VARCHAR(255)	None	No	None	Tên/tiêu đề sách, không được để trống.
isbn	VARCHAR(20)	Candidate Key	No	None	Mã số tiêu chuẩn quốc tế cho sách, duy nhất (UNIQUE).
publisher	VARCHAR(100)	None	Yes	NULL	Tên nhà xuất bản ấn hành.
publication_year	INT	None	Yes	NULL	Năm xuất bản, phải phù hợp (CHECK (publication_year >= 1500)).

3.5 Bảng book_copy (Bản sao sách vật lý)

Attribute Name	Data Type	Key Type	Nullable ?	Default Value	Business Rules / Constraints
copy_id	INT	PK	No	Auto Increment	Mã định danh duy nhất của từng bản sao vật lý.
book_id	INT	FK	No	None	Khóa ngoại tham chiếu physical_book(book_id). ON DELETE RESTRICT.
barcode	VARCHAR(50)	Candidate Key	No	None	Mã vạch dán trên cuốn sách cụ thể, duy nhất (UNIQUE). [BR-03.1]
copy_status	VARCHAR(20)	None	No	'Available'	Thuộc tập giá trị: 'Available', 'Borrowed', 'Lost', 'Damaged', 'In_Maintenance'. [BR-03.2]

shelf_location	VARCHAR(50)	None	Yes	NULL	Vị trí lưu trữ thực tế trên kệ sách (Ví dụ: 'Kệ A3-Tầng 2').

3.6 Bảng license (Giấy phép bản quyền mở)

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
license_id	INT	PK	No	Auto Increment	Mã định danh duy nhất của giấy phép bản quyền.
license_name	VARCHAR(100)	None	No	None	Tên đầy đủ của giấy phép (Ví dụ: 'Creative Commons Attribution 4.0').
license_code	VARCHAR(50)	Candidate Key	No	None	Ký hiệu viết tắt duy nhất (UNIQUE), ví dụ: 'CC BY 4.0', 'Public Domain'.
terms_description	TEXT	None	Yes	NULL	Mô tả chi tiết các điều khoản quyền hạn và nghĩa vụ sử dụng.

3.7 Bảng digital_item (Tài liệu số & Lưu trữ mở)

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
item_id	INT	PK	No	Auto Increment	Mã định danh duy nhất của tài liệu số.
license_id	INT	FK	No	None	Khóa ngoại tham chiếu license(license_id). ON DELETE

					RESTRICT. [BR-06.2]
title	VARCHAR(255)	None	No	None	Tiêu đề của bài báo, tạp chí hoặc công trình nghiên cứu số.
abstract	TEXT	None	Yes	NULL	Tóm tắt nội dung nghiên cứu của tài liệu số.
doi	VARCHAR(100)	Candidate Key	Yes	NULL	Mã định danh đối tượng số quốc tế duy nhất (UNIQUE). [BR-06.1]
file_path	VARCHAR(500)	None	No	None	Đường dẫn lưu trữ tệp tin trên hệ thống lưu trữ/server. [BR-06.1]
file_format	VARCHAR(10)	None	Yes	'PDF'	Định dạng tệp tin (Ví dụ: 'PDF', 'EPUB', 'ZIP').
access_level	VARCHAR(50)	None	No	'Public_Open_Access'	Thuộc tập giá trị: 'Public_Open_Access', 'Restricted_Campus_Only', 'Embargoed'. [BR-06.3]
embargo_end_date	DATE	None	Yes	NULL	Ngày kết thúc thời gian cấm truy cập công khai (chỉ áp dụng nếu status='Embargoed'). [BR-06.3]

3.8 Bảng borrow_transaction (Giao dịch mượn - trả sách)

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
transaction_id	INT	PK	No	Auto Increment	Mã định danh duy nhất cho mỗi giao dịch mượn trả.
member_id	INT	FK	No	None	Khóa ngoại tham chiếu member(person_id). ON DELETE RESTRICT.

copy_id	INT	FK	No	None	Khóa ngoại tham chiếu book_copy(copy_id). ON DELETE RESTRICT.
borrow_date	DATE	None	No	None	Ngày thực hiện việc mượn sách.
due_date	DATE	None	No	None	Ngày hạn trả sách (CHECK (due_date >= borrow_date)). [BR-04.2]
return_date	DATE	None	Yes	NULL	Ngày thực tế trả sách (CHECK (return_date >= borrow_date)). Nếu NULL là chưa trả. [BR-04.1, BR-04.2]

3.9 Bảng fine (Tiền phạt quá hạn / hư hỏng)

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
fine_id	INT	PK	No	Auto Increment	Mã định danh duy nhất của bản ghi tiền phạt.
transaction_id	INT	FK, Candidate Key	No	None	Khóa ngoại duy nhất (UNIQUE) tham chiếu borrow_transaction(transaction_id). ON DELETE RESTRICT. [BR-05.3]
fine_amount	DECIMAL(10,2)	None	No	0.00	Số tiền phạt phát sinh (CHECK (fine_amount >= 0.00)). [BR-05.2]
fine_reason	VARCHAR(255)	None	Yes	NULL	Lý do xử phạt (Ví dụ: 'Trả sách quá hạn 5 ngày', 'Sách bị rách trang').
fine_status	VARCHAR(20)	None	No	'Unpaid'	Giá trị chỉ gồm: 'Unpaid', 'Paid'. [BR-05.3]
payment_date	DATE	None	Yes	NULL	Ngày thủ thư xác nhận hoàn tất nộp phạt (bắt buộc khi fine_status = 'Paid'). [BR-05.3]

3.10 Bảng category (Danh mục / Chủ đề)

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
category_id	INT	PK	No	Auto Increment	Mã định danh duy nhất của

					danh mục phân loại.
category_name	VARCHAR(100)	Candidate Key	No	None	Tên danh mục/chủ đề duy nhất (UNIQUE).
description	TEXT	None	Yes	NULL	Mô tả phạm vi tri thức của danh mục/chủ đề.

3.11 Bảng book_author (Bảng liên kết Sách - Tác giả)

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
book_id	INT	PK, FK	No	None	Khóa ngoại tham chiếu physical_book(book_id). ON DELETE CASCADE.
author_id	INT	PK, FK	No	None	Khóa ngoại tham chiếu author(person_id). ON DELETE CASCADE.
author_role	VARCHAR(50)	None	Yes	'Main Author'	Vai trò tác giả, thuộc tập: 'Main Author', 'Co-Author', 'Editor', 'Translator'. [BR-07.2]

3.12 Bảng item_author (Bảng liên kết Tài liệu số - Tác giả)

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
item_id	INT	PK, FK	No	None	Khóa ngoại tham chiếu digital_item(item_id). ON DELETE CASCADE.
author_id	INT	PK, FK	No	None	Khóa ngoại tham chiếu author(person_id). ON DELETE CASCADE.
author_order	INT	None	No	1	Thứ tự xuất hiện tên tác giả trong bài báo (CHECK (author_order >= 1)). [BR-07.2]

author_role	VARCHAR(50)	None	Yes	'Principal Investigator'	Vai trò đối với công trình số (Ví dụ: 'Lead Author', 'Corresponding Author'). [BR-07.2]

3.13 Bảng item_category (Bảng liên kết Tài liệu số - Danh mục)

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
item_id	INT	PK, FK	No	None	Khóa ngoại tham chiếu digital_item(item_id). ON DELETE CASCADE. [BR-07.3]
category_id	INT	PK, FK	No	None	Khóa ngoại tham chiếu category(category_id). ON DELETE CASCADE. [BR-07.3]

4. Database Implementation (SQL Style Guide Compliant)

4.1 DDL Script (Tables, Views, Indexes, Triggers)

Kịch bản DDL dưới đây hoàn toàn đáp ứng các tiêu chuẩn SQLStyle.guide và Google SQL Style Guide: toàn bộ từ khóa SQL được VIẾT HOA, câu lệnh định dạng rõ ràng, tên bảng/cột viết bằng snake_case, tích hợp đầy đủ ràng buộc khóa chính, khóa ngoại, ràng buộc kiểm tra (CHECK) và giá trị mặc định.

Tables

```
FILE: 00_init_db.sql
1 DROP DATABASE IF EXISTS dloar_db;
2 CREATE DATABASE dloar_db
3     CHARACTER SET utf8mb4
4     COLLATE utf8mb4_unicode_ci;
```

```
 1 DROP TABLE IF EXISTS dloar_db.person;
 2
 3 CREATE TABLE dloar_db.person (
 4     person_id INT NOT NULL AUTO_INCREMENT,
 5     first_name VARCHAR(50) NOT NULL,
 6     last_name VARCHAR(50) NOT NULL,
 7     email VARCHAR(100) NOT NULL,
 8     phone_number VARCHAR(20) DEFAULT NULL,
 9     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
10     CONSTRAINT pk_person PRIMARY KEY (person_id),
```

```
    CONSTRAINT uq_person_email UNIQUE (email)
) ENGINE=InnoDB;
```

```
DROP TABLE IF EXISTS dloar_db.member;

CREATE TABLE dloar_db.member (
    person_id         INT         NOT NULL,
    member_code       VARCHAR(20) NOT NULL,
    membership_status VARCHAR(20) NOT NULL DEFAULT 'Active',
    max_borrow_limit  INT         NOT NULL DEFAULT 5,
    expiry_date       DATE        NOT NULL,

    CONSTRAINT pk_member
        PRIMARY KEY (person_id),
    CONSTRAINT uq_member_code
        UNIQUE (member_code),
    CONSTRAINT fk_member_person
        FOREIGN KEY (person_id)
        REFERENCES dloar_db.person (person_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT chk_member_status
        CHECK (membership_status IN (
            'Active', 'Expired', 'Suspended'
        )),
    CONSTRAINT chk_member_max_borrow
        CHECK (max_borrow_limit > 0)
) ENGINE=InnoDB;
```

```
DROP TABLE IF EXISTS dloar_db.author;

CREATE TABLE dloar_db.author (
    person_id   INT          NOT NULL,
    author_code VARCHAR(20)  NOT NULL,
    affiliation VARCHAR(100) DEFAULT NULL,
    orcid       VARCHAR(20)  DEFAULT NULL,
    bio         TEXT         DEFAULT NULL,

    CONSTRAINT pk_author
        PRIMARY KEY (person_id),
    CONSTRAINT uq_author_code
        UNIQUE (author_code),
    CONSTRAINT uq_author_orcid
```

```
        UNIQUE (orcid),
    CONSTRAINT fk_author_person
        FOREIGN KEY (person_id)
        REFERENCES dloar_db.person (person_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
) ENGINE=InnoDB;
```

```
DROP TABLE IF EXISTS dloar_db.physical_book;

CREATE TABLE dloar_db.physical_book (
    book_id          INT          NOT NULL AUTO_INCREMENT,
    title            VARCHAR(255) NOT NULL,
    isbn             VARCHAR(20)  NOT NULL,
    publisher        VARCHAR(100) DEFAULT NULL,
    publication_year INT          DEFAULT NULL,

    CONSTRAINT pk_physical_book
        PRIMARY KEY (book_id),
    CONSTRAINT uq_physical_book_isbn
        UNIQUE (isbn),
    CONSTRAINT chk_publication_year
        CHECK (publication_year >= 1500)
) ENGINE=InnoDB;
```

```
DROP TABLE IF EXISTS dloar_db.book_copy;

CREATE TABLE dloar_db.book_copy (
    copy_id        INT         NOT NULL AUTO_INCREMENT,
    book_id        INT         NOT NULL,
    barcode        VARCHAR(50) NOT NULL,
    copy_status    VARCHAR(20) NOT NULL DEFAULT 'Available',
    shelf_location VARCHAR(50) DEFAULT NULL,

    CONSTRAINT pk_book_copy
        PRIMARY KEY (copy_id),
    CONSTRAINT uq_book_copy_barcode
        UNIQUE (barcode),
    CONSTRAINT fk_book_copy_book
        FOREIGN KEY (book_id)
        REFERENCES dloar_db.physical_book (book_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
```

```
    CONSTRAINT chk_copy_status
        CHECK (copy_status IN (
            'Available', 'Borrowed', 'Lost',
            'Damaged', 'In_Maintenance'
        ))
) ENGINE=InnoDB;
```

```
DROP TABLE IF EXISTS dloar_db.license;

CREATE TABLE dloar_db.license (
    license_id        INT          NOT NULL AUTO_INCREMENT,
    license_name      VARCHAR(100) NOT NULL,
    license_code      VARCHAR(50)  NOT NULL,
    terms_description TEXT         DEFAULT NULL,

    CONSTRAINT pk_license
        PRIMARY KEY (license_id),
    CONSTRAINT uq_license_code
        UNIQUE (license_code)
) ENGINE=InnoDB;
```

```
DROP TABLE IF EXISTS dloar_db.digital_item;

CREATE TABLE dloar_db.digital_item (
    item_id          INT          NOT NULL AUTO_INCREMENT,
    license_id       INT          NOT NULL,
    title            VARCHAR(255) NOT NULL,
    abstract         TEXT         DEFAULT NULL,
    doi              VARCHAR(100) DEFAULT NULL,
    file_path        VARCHAR(500) NOT NULL,
    file_format      VARCHAR(10)  DEFAULT 'PDF',
    access_level     VARCHAR(50)  NOT NULL
                                  DEFAULT 'Public_Open_Access',
    embargo_end_date DATE         DEFAULT NULL,

    CONSTRAINT pk_digital_item
        PRIMARY KEY (item_id),
    CONSTRAINT uq_digital_item_doi
        UNIQUE (doi),
    CONSTRAINT fk_digital_item_license
        FOREIGN KEY (license_id)
        REFERENCES dloar_db.license (license_id)
        ON DELETE RESTRICT
```

```
        ON UPDATE CASCADE,
    CONSTRAINT chk_access_level
        CHECK (access_level IN (
            'Public_Open_Access',
            'Restricted_Campus_Only',
            'Embargoed'
        ))
) ENGINE=InnoDB;
```

```
DROP TABLE IF EXISTS dloar_db.borrow_transaction;

CREATE TABLE dloar_db.borrow_transaction (
    transaction_id INT  NOT NULL AUTO_INCREMENT,
    member_id      INT  NOT NULL,
    copy_id        INT  NOT NULL,
    borrow_date    DATE NOT NULL,
    due_date       DATE NOT NULL,
    return_date    DATE DEFAULT NULL,

    CONSTRAINT pk_borrow_transaction
        PRIMARY KEY (transaction_id),
    CONSTRAINT fk_borrow_member
        FOREIGN KEY (member_id)
        REFERENCES dloar_db.member (person_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT fk_borrow_copy
        FOREIGN KEY (copy_id)
        REFERENCES dloar_db.book_copy (copy_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT chk_borrow_due_dates
        CHECK (due_date >= borrow_date),
    CONSTRAINT chk_borrow_return_dates
        CHECK (return_date IS NULL OR return_date >= borrow_date)
) ENGINE=InnoDB;
```

```
DROP TABLE IF EXISTS dloar_db.fine;

CREATE TABLE dloar_db.fine (
    fine_id        INT            NOT NULL AUTO_INCREMENT,
    transaction_id INT            NOT NULL,
    fine_amount    DECIMAL(10, 2) NOT NULL DEFAULT 0.00,
```

```
    fine_reason    VARCHAR(255)   DEFAULT NULL,
    fine_status    VARCHAR(20)    NOT NULL DEFAULT 'Unpaid',
    payment_date   DATE           DEFAULT NULL,

    CONSTRAINT pk_fine
        PRIMARY KEY (fine_id),
    CONSTRAINT uq_fine_transaction
        UNIQUE (transaction_id),
    CONSTRAINT fk_fine_transaction
        FOREIGN KEY (transaction_id)
        REFERENCES dloar_db.borrow_transaction (transaction_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT chk_fine_amount
        CHECK (fine_amount >= 0.00),
    CONSTRAINT chk_fine_status
        CHECK (fine_status IN ('Unpaid', 'Paid'))
) ENGINE=InnoDB;
```

```
DROP TABLE IF EXISTS dloar_db.category;

CREATE TABLE dloar_db.category (
    category_id   INT          NOT NULL AUTO_INCREMENT,
    category_name VARCHAR(100) NOT NULL,
    description   TEXT         DEFAULT NULL,

    CONSTRAINT pk_category
        PRIMARY KEY (category_id),
    CONSTRAINT uq_category_name
        UNIQUE (category_name)
) ENGINE=InnoDB;
```

```
DROP TABLE IF EXISTS dloar_db.book_author;

CREATE TABLE dloar_db.book_author (
    book_id     INT         NOT NULL,
    author_id   INT         NOT NULL,
    author_role VARCHAR(50) DEFAULT 'Main Author',

    CONSTRAINT pk_book_author
        PRIMARY KEY (book_id, author_id),
    CONSTRAINT fk_ba_book
        FOREIGN KEY (book_id)
```

```
        REFERENCES dloar_db.physical_book (book_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_ba_author
        FOREIGN KEY (author_id)
        REFERENCES dloar_db.author (person_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT chk_book_author_role
        CHECK (author_role IN (
            'Main Author', 'Co-Author',
            'Editor', 'Translator'
        ))
) ENGINE=InnoDB;
```

```
DROP TABLE IF EXISTS dloar_db.item_author;

CREATE TABLE dloar_db.item_author (
    item_id      INT          NOT NULL,
    author_id    INT          NOT NULL,
    author_order INT          NOT NULL DEFAULT 1,
    author_role  VARCHAR(50) DEFAULT 'Principal Investigator',

    CONSTRAINT pk_item_author
        PRIMARY KEY (item_id, author_id),
    CONSTRAINT fk_ia_item
        FOREIGN KEY (item_id)
        REFERENCES dloar_db.digital_item (item_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_ia_author
        FOREIGN KEY (author_id)
        REFERENCES dloar_db.author (person_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT chk_item_author_order
        CHECK (author_order >= 1)
) ENGINE=InnoDB;
```

```
DROP TABLE IF EXISTS dloar_db.item_category;

CREATE TABLE dloar_db.item_category (
    item_id     INT NOT NULL,
```

```
    category_id INT NOT NULL,

    CONSTRAINT pk_item_category
        PRIMARY KEY (item_id, category_id),
    CONSTRAINT fk_ic_item
        FOREIGN KEY (item_id)
        REFERENCES dloar_db.digital_item (item_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_ic_category
        FOREIGN KEY (category_id)
        REFERENCES dloar_db.category (category_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Indexes

```
-- ======================================================
-- SYSTEM: DIGITAL LIBRARY & OPEN-ACCESS REPOSITORY
-- FILE: 02_indexes.sql
-- ======================================================

USE dloar_db;

CREATE INDEX idx_person_names
    ON dloar_db.person (last_name, first_name);

CREATE INDEX idx_borrow_member_status
    ON dloar_db.borrow_transaction (member_id, return_date);

CREATE INDEX idx_borrow_copy_status
    ON dloar_db.borrow_transaction (copy_id, return_date);

CREATE INDEX idx_book_copy_status
    ON dloar_db.book_copy (copy_status);

CREATE INDEX idx_digital_item_access
    ON dloar_db.digital_item (access_level, embargo_end_date);

CREATE INDEX idx_fine_status
    ON dloar_db.fine (fine_status);
```

Views

```sql
-- ================================================
-- SYSTEM: DIGITAL LIBRARY & OPEN-ACCESS REPOSITORY
-- FILE: 03_views.sql
-- FORMATTED FOR WORD (Consolas 12pt)
-- ================================================

USE dloar_db;

-- ------------------------------------------------
-- View 1: Thống kê giao dịch mượn đang hoạt động
-- ------------------------------------------------
CREATE OR REPLACE VIEW dloar_db.vw_active_loans AS
SELECT
  bt.transaction_id,
  m.member_code,
  CONCAT(p.last_name, ' ', p.first_name) AS member_name,
  p.email,
  pb.title AS book_title,
  bc.barcode,
  bt.borrow_date,
  bt.due_date,
  DATEDIFF(CURRENT_DATE, bt.due_date) AS days_overdue
FROM dloar_db.borrow_transaction bt
JOIN dloar_db.member m
  ON bt.member_id = m.person_id
JOIN dloar_db.person p
  ON m.person_id = p.person_id
JOIN dloar_db.book_copy bc
  ON bt.copy_id = bc.copy_id
JOIN dloar_db.physical_book pb
  ON bc.book_id = pb.book_id
WHERE bt.return_date IS NULL;

-- ------------------------------------------------
-- View 2: Tổng hợp nợ phạt chưa thanh toán
-- ------------------------------------------------
CREATE OR REPLACE VIEW dloar_db.vw_unpaid_fines_summary AS
SELECT
  m.person_id AS member_id,
  m.member_code,
  CONCAT(p.last_name, ' ', p.first_name) AS member_name,
  p.email,
  COUNT(f.fine_id) AS unpaid_fines_count,
  SUM(f.fine_amount) AS total_unpaid_amount
```

```sql
FROM dloar_db.fine f
JOIN dloar_db.borrow_transaction bt
  ON f.transaction_id = bt.transaction_id
JOIN dloar_db.member m
  ON bt.member_id = m.person_id
JOIN dloar_db.person p
  ON m.person_id = p.person_id
WHERE f.fine_status = 'Unpaid'
GROUP BY
  m.person_id,
  m.member_code,
  p.last_name,
  p.first_name,
  p.email;

-- ------------------------------------------------
-- View 3: Danh mục tài liệu số mở công khai
-- ------------------------------------------------
CREATE OR REPLACE VIEW dloar_db.vw_public_repository_catalog AS
SELECT
  di.item_id,
  di.title,
  di.doi,
  l.license_name,
  l.license_code,
  di.file_path,
  GROUP_CONCAT(
    c.category_name SEPARATOR ', '
  ) AS categories
FROM dloar_db.digital_item di
JOIN dloar_db.license l
  ON di.license_id = l.license_id
LEFT JOIN dloar_db.item_category ic
  ON di.item_id = ic.item_id
LEFT JOIN dloar_db.category c
  ON ic.category_id = c.category_id
WHERE di.access_level = 'Public_Open_Access'
   OR (di.access_level = 'Embargoed'
       AND di.embargo_end_date < CURRENT_DATE)
GROUP BY
  di.item_id,
  di.title,
  di.doi,
  l.license_name,
  l.license_code,
```

```
90   di.file_path;
```

Triggers

```
 1 -- ================================================
 2 -- SYSTEM: DIGITAL LIBRARY & OPEN-ACCESS REPOSITORY
 3 -- FILE: 04_triggers.sql
 4 -- FORMATTED FOR WORD (Consolas 12pt)
 5 -- ================================================
 6
 7 USE dloar_db;
 8
 9 DELIMITER //
10
11 -- ------------------------------------------------
12 -- Trigger 1: Tự động cập nhật 'Borrowed' khi mượn sách
13 -- ------------------------------------------------
14 DROP TRIGGER IF EXISTS dloar_db.trg_after_borrow_insert //
15 CREATE TRIGGER dloar_db.trg_after_borrow_insert
16 AFTER INSERT ON dloar_db.borrow_transaction
17 FOR EACH ROW
18 BEGIN
19   UPDATE dloar_db.book_copy
20   SET copy_status = 'Borrowed'
21   WHERE copy_id = NEW.copy_id;
22 END //
23
24 -- ------------------------------------------------
25 -- Trigger 2: Cập nhật 'Available' & phạt quá hạn khi trả
26 -- ------------------------------------------------
27 DROP TRIGGER IF EXISTS dloar_db.trg_after_borrow_update_return //
28 CREATE TRIGGER dloar_db.trg_after_borrow_update_return
29 AFTER UPDATE ON dloar_db.borrow_transaction
30 FOR EACH ROW
31 BEGIN
32   DECLARE v_days_overdue INT;
33   DECLARE v_penalty_rate DECIMAL(10,2) DEFAULT 5000.00;
34
35   IF NEW.return_date IS NOT NULL
36      AND OLD.return_date IS NULL THEN
37
38     -- Cập nhật Available nếu bản sao chưa hỏng/mất
39     UPDATE dloar_db.book_copy
40     SET copy_status = 'Available'
41     WHERE copy_id = NEW.copy_id
```

```
      AND copy_status = 'Borrowed';

    -- Tính tiền phạt nếu trả quá hạn
    IF NEW.return_date > NEW.due_date THEN
      SET v_days_overdue =
        DATEDIFF(NEW.return_date, NEW.due_date);

      INSERT INTO dloar_db.fine (
        transaction_id,
        fine_amount,
        fine_reason,
        fine_status
      )
      VALUES (
        NEW.transaction_id,
        v_days_overdue * v_penalty_rate,
        CONCAT('Trả quá hạn ', v_days_overdue, ' ngày'),
        'Unpaid'
      );
    END IF;
  END IF;
END //

-- ------------------------------------------------------
-- Trigger 3: Phạt đền bù khi sách bị 'Damaged' hoặc 'Lost'
-- ------------------------------------------------------
DROP TRIGGER IF EXISTS dloar_db.trg_after_copy_status_update_fine
//
CREATE TRIGGER dloar_db.trg_after_copy_status_update_fine
AFTER UPDATE ON dloar_db.book_copy
FOR EACH ROW
BEGIN
  DECLARE v_active_trans_id INT;

  IF NEW.copy_status IN ('Damaged', 'Lost')
     AND OLD.copy_status != NEW.copy_status THEN

    -- Tìm giao dịch mượn gần nhất của bản sao này
    SELECT transaction_id INTO v_active_trans_id
    FROM dloar_db.borrow_transaction
    WHERE copy_id = NEW.copy_id
    ORDER BY borrow_date DESC
    LIMIT 1;

    IF v_active_trans_id IS NOT NULL THEN
```

```
      INSERT INTO dloar_db.fine (
        transaction_id,
        fine_amount,
        fine_reason,
        fine_status
      )
      VALUES (
        v_active_trans_id,
        IF(NEW.copy_status = 'Lost', 200000.00, 50000.00),
        CONCAT(
          'Tiền phạt đền bù do sách bị ',
          IF(NEW.copy_status = 'Lost', 'MẤT', 'HƯ HỎNG')
        ),
        'Unpaid'
      );
    END IF;
  END IF;
END //

  DELIMITER ;
```

4.2 Advanced Queries & Performance Test Cases

Dưới đây là 10 truy vấn phức tạp thể hiện đầy đủ các kỹ thuật truy vấn nâng cao (Phép nối phức hợp JOIN, Truy vấn con Subqueries, Hàm gom nhóm Aggregations, Hàm cửa sổ Window Functions, Bảng tạm CTEs):

```
-- ================================================
-- SYSTEM: DIGITAL LIBRARY & OPEN-ACCESS REPOSITORY
-- FILE: 05_queries.sql (Advanced Queries)
-- FORMATTED FOR WORD (Consolas 12pt)
-- ================================================

USE dloar_db;

-- ------------------------------------------------
-- Truy vấn 1: Báo cáo thực thể trùng lặp (Độc giả & Tác giả)
-- ------------------------------------------------
WITH member_borrow_stats AS (
  SELECT
    member_id,
    COUNT(*) AS total_borrows
  FROM dloar_db.borrow_transaction
```

```sql
  GROUP BY member_id
),
author_item_stats AS (
  SELECT
    author_id,
    COUNT(*) AS total_digital_publications
  FROM dloar_db.item_author
  GROUP BY author_id
)
SELECT
  p.person_id,
  CONCAT(p.last_name, ' ', p.first_name) AS full_name,
  p.email,
  m.member_code,
  a.author_code,
  a.affiliation,
  COALESCE(mbs.total_borrows, 0) AS total_borrows,
  COALESCE(ais.total_digital_publications, 0)
    AS total_digital_publications
FROM dloar_db.person p
JOIN dloar_db.member m
  ON p.person_id = m.person_id
JOIN dloar_db.author a
  ON p.person_id = a.person_id
LEFT JOIN member_borrow_stats mbs
  ON m.person_id = mbs.member_id
LEFT JOIN author_item_stats ais
  ON a.person_id = ais.author_id
ORDER BY
  total_digital_publications DESC,
  total_borrows DESC;

-- ------------------------------------------------
-- Truy vấn 2: Xếp hạng các đầu sách vật lý mượn nhiều nhất
-- ------------------------------------------------
WITH book_usage AS (
  SELECT
    pb.book_id,
    pb.title,
    pb.isbn,
    COUNT(bt.transaction_id) AS borrow_count
  FROM dloar_db.physical_book pb
  JOIN dloar_db.book_copy bc
    ON pb.book_id = bc.book_id
  JOIN dloar_db.borrow_transaction bt
```

```sql
    ON bc.copy_id = bt.copy_id
  GROUP BY
    pb.book_id,
    pb.title,
    pb.isbn
)
SELECT
  bu.book_id,
  bu.title,
  bu.isbn,
  bu.borrow_count,
  DENSE_RANK() OVER (
    ORDER BY bu.borrow_count DESC
  ) AS overall_rank
FROM book_usage bu
LIMIT 10;

-- ------------------------------------------------------
-- Truy vấn 3: Phân tích vi phạm quá hạn & tiền phạt
-- ------------------------------------------------------
SELECT
  m.membership_status,
  COUNT(DISTINCT m.person_id) AS total_members,
  COUNT(f.fine_id) AS total_fine_records,
  SUM(
    CASE WHEN f.fine_status = 'Unpaid'
         THEN f.fine_amount ELSE 0
    END
  ) AS total_unpaid_amount,
  SUM(
    CASE WHEN f.fine_status = 'Paid'
         THEN f.fine_amount ELSE 0
    END
  ) AS total_collected_amount
FROM dloar_db.member m
JOIN dloar_db.borrow_transaction bt
  ON m.person_id = bt.member_id
JOIN dloar_db.fine f
  ON bt.transaction_id = f.transaction_id
GROUP BY
  m.membership_status;

-- ------------------------------------------------------
-- Truy vấn 4: Kiểm tra điều kiện mượn sách mới [BR-02.3]
-- ------------------------------------------------------
```

```sql
WITH active_loans AS (
  SELECT
    member_id,
    COUNT(*) AS active_loan_count
  FROM dloar_db.borrow_transaction
  WHERE return_date IS NULL
  GROUP BY member_id
),
unpaid_fines AS (
  SELECT
    bt.member_id,
    SUM(f.fine_amount) AS total_unpaid_fine
  FROM dloar_db.fine f
  JOIN dloar_db.borrow_transaction bt
    ON f.transaction_id = bt.transaction_id
  WHERE f.fine_status = 'Unpaid'
  GROUP BY bt.member_id
)
SELECT
  m.person_id,
  m.member_code,
  CONCAT(p.last_name, ' ', p.first_name) AS member_name,
  m.membership_status,
  m.max_borrow_limit,
  COALESCE(al.active_loan_count, 0) AS current_active_loans,
  COALESCE(uf.total_unpaid_fine, 0) AS total_unpaid_fine,
  CASE
    WHEN m.membership_status != 'Active'
      THEN 'BLOCKED: Tài khoản không hoạt động'
    WHEN m.expiry_date < CURRENT_DATE
      THEN 'BLOCKED: Thẻ đã hết hạn'
    WHEN COALESCE(al.active_loan_count, 0) >= m.max_borrow_limit
      THEN 'BLOCKED: Đạt giới hạn mượn tối đa'
    WHEN COALESCE(uf.total_unpaid_fine, 0) > 0
      THEN 'BLOCKED: Đang có khoản nợ tiền phạt'
    ELSE 'ELIGIBLE: Đủ điều kiện mượn'
  END AS borrow_eligibility_status
FROM dloar_db.member m
JOIN dloar_db.person p
  ON m.person_id = p.person_id
LEFT JOIN active_loans al
  ON m.person_id = al.member_id
LEFT JOIN unpaid_fines uf
  ON m.person_id = uf.member_id;

```

```sql
-- ------------------------------------------------------
-- Truy vấn 5: Phân bổ tài liệu số theo giấy phép & truy cập
-- ------------------------------------------------------
SELECT
  l.license_code,
  l.license_name,
  SUM(
    CASE WHEN di.access_level = 'Public_Open_Access'
         THEN 1 ELSE 0
    END
  ) AS public_access_count,
  SUM(
    CASE WHEN di.access_level = 'Restricted_Campus_Only'
         THEN 1 ELSE 0
    END
  ) AS campus_only_count,
  SUM(
    CASE WHEN di.access_level = 'Embargoed'
         THEN 1 ELSE 0
    END
  ) AS embargoed_count,
  COUNT(di.item_id) AS total_items
FROM dloar_db.license l
LEFT JOIN dloar_db.digital_item di
  ON l.license_id = di.license_id
GROUP BY
  l.license_id,
  l.license_code,
  l.license_name
ORDER BY
  total_items DESC;

-- ------------------------------------------------------
-- Truy vấn 6: Tài liệu số (Embargoed) đã hết hạn khóa [BR-06.3]
-- ------------------------------------------------------
SELECT
  di.item_id,
  di.title,
  di.doi,
  di.access_level,
  di.embargo_end_date,
  DATEDIFF(
    CURRENT_DATE, di.embargo_end_date
  ) AS days_since_embargo_expired
FROM dloar_db.digital_item di
```

```sql
WHERE di.access_level = 'Embargoed'
  AND di.embargo_end_date IS NOT NULL
  AND di.embargo_end_date < CURRENT_DATE;

-- ------------------------------------------------------
-- Truy vấn 7: Tác giả đóng góp nhiều nhất cho Kho lưu trữ mở
-- ------------------------------------------------------
SELECT
  a.person_id AS author_id,
  a.author_code,
  CONCAT(p.last_name, ' ', p.first_name) AS author_name,
  a.affiliation,
  a.orcid,
  COUNT(ia.item_id) AS total_publications,
  MIN(ia.author_order) AS highest_authorship_order
FROM dloar_db.author a
JOIN dloar_db.person p
  ON a.person_id = p.person_id
JOIN dloar_db.item_author ia
  ON a.person_id = ia.author_id
GROUP BY
  a.person_id,
  a.author_code,
  p.last_name,
  p.first_name,
  a.affiliation,
  a.orcid
HAVING
  COUNT(ia.item_id) >= 1
ORDER BY
  total_publications DESC;

-- ------------------------------------------------------
-- Truy vấn 8: Thống kê bản sao sách (book_copy) theo trạng thái
-- ------------------------------------------------------
SELECT
  pb.book_id,
  pb.title,
  pb.isbn,
  COUNT(bc.copy_id) AS total_copies,
  SUM(
    CASE WHEN bc.copy_status = 'Available'
         THEN 1 ELSE 0
    END
  ) AS available_copies,
```

```sql
  SUM(
    CASE WHEN bc.copy_status = 'Borrowed'
         THEN 1 ELSE 0
    END
  ) AS borrowed_copies,
  SUM(
    CASE WHEN bc.copy_status = 'Damaged'
         THEN 1 ELSE 0
    END
  ) AS damaged_copies,
  SUM(
    CASE WHEN bc.copy_status = 'Lost'
         THEN 1 ELSE 0
    END
  ) AS lost_copies,
  SUM(
    CASE WHEN bc.copy_status = 'In_Maintenance'
         THEN 1 ELSE 0
    END
  ) AS maintenance_copies
FROM dloar_db.physical_book pb
LEFT JOIN dloar_db.book_copy bc
  ON pb.book_id = bc.book_id
GROUP BY
  pb.book_id,
  pb.title,
  pb.isbn;

-- ------------------------------------------------
-- Truy vấn 9: Doanh thu tiền phạt theo từng tháng
-- ------------------------------------------------
SELECT
  DATE_FORMAT(f.payment_date, '%Y-%m') AS payment_month,
  COUNT(f.fine_id) AS total_fines_paid,
  SUM(f.fine_amount) AS total_revenue
FROM dloar_db.fine f
WHERE f.fine_status = 'Paid'
  AND f.payment_date IS NOT NULL
GROUP BY
  DATE_FORMAT(f.payment_date, '%Y-%m')
ORDER BY
  payment_month DESC;

-- ------------------------------------------------
-- Truy vấn 10: Tài liệu số thuộc nhiều hơn 1 danh mục
```

```sql
-- ------------------------------------------------
SELECT
  di.item_id,
  di.title,
  di.doi,
  COUNT(ic.category_id) AS category_count,
  GROUP_CONCAT(
    c.category_name
    ORDER BY c.category_name
    SEPARATOR ' | '
  ) AS categories_list
FROM dloar_db.digital_item di
JOIN dloar_db.item_category ic
  ON di.item_id = ic.item_id
JOIN dloar_db.category c
  ON ic.category_id = c.category_id
GROUP BY
  di.item_id,
  di.title,
  di.doi
HAVING
  COUNT(ic.category_id) > 1
ORDER BY
  category_count DESC;
```

Performance Test Case & Optimization Strategy (Kịch bản kiểm thử hiệu năng & Tối ưu hóa)

- **Kịch bản kiểm thử:** Thực thi truy vấn tra cứu lịch sử mượn sách của độc giả dựa trên member_id và trạng thái mượn return_date IS NULL.
- **Phân tích kế hoạch thực thi (EXPLAIN Analysis):**
  - Khi chưa có Chỉ mục (Index): Cơ sở dữ liệu thực hiện Quét toàn bộ bảng (Full Table Scan) trên bảng borrow_transaction với chi phí cost cao và thời gian phản hồi lớn khi dữ liệu đạt hàng triệu bản ghi.
  - Khi đã thêm Chỉ mục Phức hợp (Composite Index): idx_borrow_member_status trên tập thuộc tính (member_id, return_date).
  - Kết quả sau tối ưu: Phép truy vấn chuyển sang dạng Index Range Scan, giảm thời gian đọc đĩa, trả về kết quả dưới 10ms, đáp ứng yêu cầu hiệu năng ISO/IEC 25010.

5. Verification & Security

5.1 Test cases showing constraints prevent bad data insertion

```sql
-- ============================================================
-- 5.1 KỊCH BẢN KIỂM THỬ RÀNG BUỘC TOÀN VẸN
-- (CONSTRAINT TEST CASES)
-- ============================================================

-- Test Case 1: Thêm Email trùng lặp vào bảng PERSON
-- (Kiểm tra uq_person_email)
-- Kết quả: FAIL (ERROR 1062: Duplicate entry)
INSERT INTO person (
    first_name,
    last_name,
    email
)
VALUES (
    'An',
    'Nguyễn',
    'n25dcat107@gmail.student.ptithcm.edu.vn'
);


-- Test Case 2: Thêm Member có giới hạn mượn <= 0
-- (Kiểm tra chk_member_max_borrow)
-- Kết quả: FAIL (ERROR 3819: Check constraint violated)
INSERT INTO member (
    person_id,
    member_code,
    membership_status,
    max_borrow_limit,
    expiry_date
)
VALUES (
    1,
    'MEM999',
    'Active',
    0,
    '2026-12-31'
);


-- Test Case 3: Thêm Giao dịch mượn có due_date < borrow_date
-- (Kiểm tra chk_borrow_due_dates)
-- Kết quả: FAIL (ERROR 3819: Check constraint violated)
```

```sql
INSERT INTO borrow_transaction (
    member_id,
    copy_id,
    borrow_date,
    due_date
)
VALUES (
    1,
    1,
    '2026-08-20',
    '2026-08-10'
);


-- Test Case 4: Xóa MEMBER khi đang có lịch sử mượn sách
-- (Kiểm tra ON DELETE RESTRICT)
-- Kết quả: FAIL (ERROR 1451: Cannot delete parent row)
DELETE FROM member
WHERE person_id = 1;
```

5.2 Role-Based Access Control (RBAC) definition (GRANT/REVOKE statements)

```sql
-- ============================================================
-- 5.2 THIẾT LẬP BẢO MẬT VÀ PHÂN QUYỀN TRUY CẬP (RBAC)
-- ============================================================

-- 1. Tạo các vai trò (Roles)
CREATE ROLE IF NOT EXISTS 'role_admin';
CREATE ROLE IF NOT EXISTS 'role_librarian';
CREATE ROLE IF NOT EXISTS 'role_reader';


-- 2. Phân quyền cho Vai trò Admin (Toàn quyền hệ thống)
GRANT ALL PRIVILEGES ON dloar_db.*
    TO 'role_admin';


-- 3. Phân quyền cho Vai trò Thủ thư (Librarian)
-- Quản lý sách, độc giả, giao dịch mượn/trả, tiền phạt
GRANT SELECT, INSERT, UPDATE
    ON dloar_db.physical_book
    TO 'role_librarian';

GRANT SELECT, INSERT, UPDATE
    ON dloar_db.book_copy
    TO 'role_librarian';

GRANT SELECT, INSERT, UPDATE
    ON dloar_db.member
    TO 'role_librarian';

GRANT SELECT, INSERT, UPDATE
    ON dloar_db.borrow_transaction
    TO 'role_librarian';

GRANT SELECT, INSERT, UPDATE
    ON dloar_db.fine
    TO 'role_librarian';

GRANT SELECT
    ON dloar_db.vw_active_loans
    TO 'role_librarian';

GRANT SELECT
    ON dloar_db.vw_unpaid_fines_summary
    TO 'role_librarian';

```

```
46
47 -- 4. Phân quyền cho Vai trò Độc giả/Sinh viên (Reader)
48 -- Chỉ có quyền xem danh mục tài liệu mở và tra cứu sách
49 GRANT SELECT
50     ON dloar_db.vw_public_repository_catalog
51     TO 'role_reader';
52
53 GRANT SELECT (
54     book_id, title, isbn,
55     publisher, publication_year
56 )
57     ON dloar_db.physical_book
58     TO 'role_reader';
59
60 GRANT SELECT (
61     copy_id, book_id,
62     copy_status, shelf_location
63 )
64     ON dloar_db.book_copy
65     TO 'role_reader';
66
67
68 -- 5. Áp dụng cấp Vai trò cho Người dùng cụ thể
69 CREATE USER IF NOT EXISTS 'librarian_user'@'localhost'
70     IDENTIFIED BY 'LibPass123!';
71
72 GRANT 'role_librarian'
73     TO 'librarian_user'@'localhost';
74
75 SET DEFAULT ROLE 'role_librarian'
76     TO 'librarian_user'@'localhost';
77
78 FLUSH PRIVILEGES;
```

Mục lục

A. Project Identity 1

B. Report structure 1

1. Introduction & Project Scope (Adapted from ISO/IEC/IEEE 29148) 1

1.1 System Objective 1

1.1.1 Problem Statement 1

1.1.2 Rationale 2

1.1.3 System Objective 2
1.1.4 Project Objectives 3
1.1.5 Entity Overview 3
1.1.5 Phân hệ Thành viên & Tác giả 3
1.1.5 Phân hệ Tài liệu vật lý 4
1.1.5 Phân hệ Tài liệu số & Lưu trữ mở 4
1.1.5 Phân hệ Mượn – Trả & Tiền phạt 5
1.1.5 Phân hệ Liên kết & Phân loại 5
1.2 Business Rules & Constraints 6
2. Database Design (ISO/IEC 19505 / IE Standards) 10
2.1 Conceptual Model (ER/EER Diagram) 10
2.2 Logical Schema Mapping (Chuyển đổi sang Lược đồ Quan hệ) 11
2.3 Normalization Verification (Chứng minh chuẩn hóa 1NF -> BCNF) 20
3. Data Dictionary (Adapted from ISO/IEC 11179) 24
3.1 Bảng person (Thực thể cha Supertype) 25
3.2 Bảng member (Thực thể con Subtype - Độc giả) 25
3.3 Bảng author (Thực thể con Subtype - Tác giả) 26
3.4 Bảng physical_book (Đầu sách vật lý) 27
3.5 Bảng book_copy (Bản sao sách vật lý) 27
3.6 Bảng license (Giấy phép bản quyền mở) 28
3.7 Bảng digital_item (Tài liệu số & Lưu trữ mở) 28
3.8 Bảng borrow_transaction (Giao dịch mượn - trả sách) 29
3.9 Bảng fine (Tiền phạt quá hạn / hư hỏng) 30
3.10 Bảng category (Danh mục / Chủ đề) 30
3.11 Bảng book_author (Bảng liên kết Sách - Tác giả) 31
3.12 Bảng item_author (Bảng liên kết Tài liệu số - Tác giả) 31
3.13 Bảng item_category (Bảng liên kết Tài liệu số - Danh mục) 32
4. Database Implementation (SQL Style Guide Compliant) 32
4.1 DDL Script (Tables, Views, Indexes, Triggers) 32

4.2 Advanced Queries & Performance Test Cases 44
5. Verification & Security 52
5.1 Test cases showing constraints prevent bad data insertion 52
5.2 Role-Based Access Control (RBAC) definition (GRANT/REVOKE statements) 53